

구조 설명 문서

결과물 URL

<https://final-spa-project.vercel.app/>

2-1. 기능 구현 부문

- 구현한 부문
 - 기본 구조 완성도
 - 추가 미션 [선택 1]
 - 추가 미션 [선택 2]
 - 추가 미션 [선택 3]
 - 추가 미션 [선택 4]

2-2. 구조 설명 서술

문항A - 14주차 누적 데이터 흐름 추적

1. 목록 화면에서 특정 영화 카드를 선택

```
// views/MoviesView.vue

<RouterLink
  :to="/movies/${movie.id}"
  class="stretched-link"
  :aria-label="${movie.title} 상세 정보 보기"
></RouterLink>
```

1. 사용자가 목록 화면에서 특정 영화 카드를 누르면, `<RouterLink>` 가 활성화 된다.
2. 라우터 링크는 다른 페이지로 이동을 처리하는 네비게이션 역할을 한다.
3. 이 `<RouterLink>` 는 `v-for="movie in getPagedMovies()"` 로 그려지는 각 카드 안에 들어 있다. 따라서 여기서의 movie는 getPagedMovies()가 잘라 반환한 현재 페이지의

영화이고, 그 movie.id(TMDB API가 제공하는 고유 번호)가

`:to="/movies/${movie.id}"`의 URL 뒤에 포함되어 들어간다.

4. 사용자가 선택한 영화의 데이터가 url로 변환되어 이어질 페이지의 주소를 지정한다.

2. 경로 매칭

```
// router/index.js

{
  path: "/movies/:id",
  name: "movie-detail",
  component: MovieDetailView,
},
```

1. `<RouterLink>`를 통해 다른 페이지로 이동을 하고자 할 때, `router/index.js`에 있는 라우터 규칙이 참고 된다. 앞선 과정에서 사용자가 확인하고자하는 영화를 눌렀을 때, 이 규칙을 참고한다.
2. `:id`는 변하는 파라미터가 들어가는 구간으로, `/movies/` 뒤에 오는 값이 무엇이든 `id`라는 이름표에 값이 담긴다. `movie.id`가 1이라면 `id = "1"`이 된다.
3. 이후, 경로가 매칭되면 `MovieDetailView` 컴포넌트를 불러온다.

3. 번호 추출

```
// views/MovieDetailView.vue

const store = useMovieStore();
const route = useRoute();

onMounted(() => {
  const movieId = route.params.id;
  store.fetchMovieDetail(movieId);
});
```

1. 라우터에서 `MovieDetailView`이 호출됨에 따라 `views/MovieDetailView.vue`이 실행된다.

2. route에는 useRoute()가 가지고 있는 현재 라우트 정보를 담는다.
 3. 그리고 컴포넌트가 처음 화면에 올라오면(마운트되면) `route.params.id` 를 꺼내와서, `movieId` 변수에 담는다. 1번 과정의 `movie.id`와 동일한 값이며, URL에서 읽어온 값이므로 문자열 "1"의 형태이다. (`movie.id`가 10이라면)
 4. 그렇게 꺼내온 영화의 고유 번호 `movieId`를 store가 가지고 있는 액션 `fetchMovieDetail()`로 넘긴다.
4. 비동기 통신 / 데이터 요청

```
// stores/movieStore.js

const fetchMovieDetail = async (movieId) => {
  isLoading.value = true;
  errorMessage.value = "";
  selectedMovie.value = null;

  try {
    const API_KEY = "454c8a94d8af51c47824cb1abf74c53c";
    const url = `https://api.themoviedb.org/3/movie/${movieId}`;

    const response = await axios.get(url, {
      params: {
        api_key: API_KEY,
        language: "ko-KR",
      },
    });
    selectedMovie.value = response.data;
  } catch (error) {
    if (error.response && error.response.status === 404) {
      errorMessage.value = "존재하지 않거나 삭제된 영화 정보입니다.";
    } else {
      errorMessage.value = "서버 통신 중 에러가 발생했습니다";
    }
  }
}
```

```

다.";
    }
  } finally {
    isLoading.value = false;
  }
};

```

1. axios를 통한 비동기 통신을 수행하는 부분으로, isLoading을 true로 바꾸고 selectedMovie를 null로 초기화함으로써 이전 데이터를 초기화 한다.
 2. `views/MovieDetailView.vue` 에서 `fetchMovieDetail()`가 `movieId`를 인자로 받아 호출되면 TMDB의 영화의 상세 정보를 조회해 볼 수 있는 url에 `movieId`를 덧붙여서 get 요청을 보낼 url을 만든다.
 3. 이후, `await axios.get()`으로 `movieId`가 포함된 URL에 비동기 요청을 보낸다.
 4. 응답 도착 시, `selectedMovie.value`에 반환된 데이터를 할당한다. `selectedMovie`는 store에 `ref(null)`로 선언된 전역 상태이기 때문에, 이 할당이 곧 다음 단계의 화면 갱신을 일으키는 트리거가 된다.
 5. 반환이 도착하면 `finally`에서 `isLoading`을 `false`로 바꿔 비동기 통신을 마친다.
5. 서버 응답 데이터를 HTML 템플릿에 결합

```

// views/MovieDetailView.vue

<main v-if="store.selectedMovie" class="detail-page">
  ...
  <h1 class="movie-main-title">{{ store.selectedMovie?.title }}</h1>
  <span class="badge rating">
    ★ {{ store.selectedMovie?.vote_average.toFixed(1) }} /
10
  </span>
  <span
    v-for="genre in store.selectedMovie?.genres"
    :key="genre.id"
    class="genre-tag"

```

```

    >
      {{ genre.name }}
    </span>
    <p class="synopsis-text">
      {{ store.selectedMovie?.overview || '줄거리가 없습니다.' }}
    </p>
    ...
  </main>
  <div v-else-if="store.isLoading" class="full-screen-loading-gate">
    ... 생략 (로딩 화면 렌더링)
  </div>
  <div v-else-if="store.errorMessage" class="full-screen-error-gate">
    ... 생략 (에러 화면 렌더링)
  </div>

```

1. 앞선 단계에서도 확인할 수 있듯이, selectedMovie는 store에 ref(null)로 선언된 전역 상태이다. `selectedMovie.value = response.data;` 로 값이 할당되는 순간, 반응형이기 때문에 이 값을 참조하던 컴포넌트들이 다시 렌더링 된다.
2. 화면은 selectedMovie의 상태에 따라 세 개로 분기한다.
 - a. response.data가 selectedMovie.value에 할당되기 전에는 selectedMovie가 null이므로 `v-if="store.selectedMovie"` 가 거짓이 되어 하단 태그가 렌더링되지 않는다. 대신, isLoading이 true이므로 `v-else-if="store.isLoading"` 에 의해 로딩 화면이 보인다.
 - b. response.data가 도착해서 selectedMovie.value에 담기면 `v-if="store.selectedMovie"` 가 참이 되어 영화에 대한 상세 내역이 렌더링 된다.
 - c. 통신에 실패하여 errorMessage가 채워졌다면 `v-else-if="store.errorMessage"` 가 참이 되어 에러 화면이 렌더링 된다.
3. 템플릿 코드에서는 전역 상태 store.selectedMovie의 각 속성을 HTML에 결합한다. `{{ store.selectedMovie.title }}` 로 제목을, vote_average로 평점을, genres는 v-for로 돌려 장르 태그를 그리고, overview로 줄거리를 출력한다.

문항B - 구현 코드 연산 논리 서술



정렬, 검색, 찜 목록, 페이지네이션 모두 서술했습니다!

• 추가 미션 [선택 1] - 정렬

```
// stores/movieStore.js

const sortBy = ref("title");

const sortMovies = () => {
  if (sortBy.value === "title") {
    movies.value.sort((a, b) => a.title.localeCompare(b.title, "ko"));
  } else if (sortBy.value === "release") {
    movies.value.sort((a, b) =>
      (b.release_date || "").localeCompare(a.release_date || ""));
  } else if (sortBy.value === "rating") {
    movies.value.sort((a, b) => b.vote_average - a.vote_average);
  }
};
```

◦ sortBy

- 현재 어떤 기준으로 정렬 되어 있는지 를 담는 변수이다. ref()로 감쌌기 때문에, Vue의 시스템이 해당 변수를 계속 추적한다.
- 초깃값은 "title"로 설정하여, 제목 순 정렬이 기본이 되도록 한다. 이후 html 태그로부터 "release"나 "rating"을 전달받는다면 해당 값을 sortBy에 저장하도록 한다. (변수를 통해 정렬을 수행)

◦ sortMovies()

- 함수가 호출되면 앞서 선언해둔 sortBy의 값을 추적해, 정렬을 할 기준을 정한다.

- sortBy의 value가 "title"이면 제목, "release"면 개봉일, "rating"이면 평점 순으로 정렬하도록 한다.
- 내부에선 sort() 함수를 이용하여 movies 내의 값들을 인플레이스 방식으로 정렬한다. 또한, (a, b)가 돌려주는 숫자의 부호로 두 가지 요소 중 어느 게 더 우선인지 파악한다. (음수면 a가 앞, 양수면 b가 앞, 0인 경우에는 순서 유지)
- movies가 반응형으로 선언되었기 때문에, movies 내부의 순서가 바뀌면 이를 v-for로 그리던 화면이 자동으로 다시 렌더링된다.
- **제목 순 정렬**
 - `a.title.localeCompare(b.title, "ko")` 는 movies 내의 두 영화의 제목을 사전 순으로 비교한다. a가 사전 순으로 앞서면 음수를 반환하므로, movies는 오름차순으로 재정렬된다.
 - 또, 두번째 인자에 "ko"를 명시함으로써 한국어 사전 순서를 보장한다.
- **개봉일 순 정렬**
 - `(b.release_date || "").localeCompare(a.release_date || "")` release_date 는 "YYYY-MM-DD"의 형태로 이루어져 있기 때문에 사전식 비교를 적용할 수 있다. b를 기준으로 a를 비교하도록 하여 최신 날짜가 앞으로오는 내림차순을 구현한다.
 - 또, || 는 정보에 개봉일이 없을 수도 있음을 염두에 두어(undefined인 경우) 해당 코드 호출시 발생할 수 있는 문제를 방지한다.
- **평점 순**
 - `b.vote_average - a.vote_average` 로 식을 설정해둘 경우, rating이 높은 쪽이 음수를 반환하기에 앞 순서로 이동한다. 이를 통해 movies를 내림차순으로 정렬한다.

```
// stores/movieStore.js

const setSortBy = (key) => {
  sortBy.value = key;
  sortMovies();
};
```

◦ **setSortBy()**

- html에서 버튼을 클릭할 시 호출되는 이벤트 핸들러이다.
- 해당 함수가 호출되면, 클릭한 버튼이 넘겨준 인자(title/release/rating)로 변수의 값을 교체한다.
- 이후, sortMovies()를 호출하여 사용자가 선택한 sortBy에 따라 movies의 배열을 재배치한다.

```
// views/MoviesView.vue

<div class="sort-bar">
  <button
    @click="store.setSortBy('title')"
    :class="{ active: store.sortBy === 'title' }"
    class="sort-btn"
  >
    제목 순
  </button>
  <button
    @click="store.setSortBy('release')"
    :class="{ active: store.sortBy === 'release' }"
    class="sort-btn"
  >
    개봉일 순
  </button>
  <button
    @click="store.setSortBy('rating')"
    :class="{ active: store.sortBy === 'rating' }"
    class="sort-btn"
  >
    평점 순
  </button>
</div>
```

- <template> 태그 안에 앞서 구현한 메소드를 실행할 수 있는 버튼들을 생성한다.

- 사용자가 버튼을 누르면, `@click="store.setSortBy({'title'/'release'/'rating'})` 을 통해 이벤트 핸들러로 문자열을 인자로 전달해 sortBy의 값을 변환시킨다.
- `:class="{ active: store.sortBy === {'title'/'release'/'rating'}}"` sortBy가 사용자가 선택한 버튼의 인자와 같을 때에만 active 클래스를 부여한다. 사용자가 현재 페이지가 어떤 기준으로 정렬중인지 시각적으로 알 수 있게끔 해준다.

• 추가 미션 [선택 2] - 검색

- 검색의 경우에는 사용자가 입력한 검색어를 주소창으로 실어 보내고 결과 페이지가 그 검색어를 받아 `filter` 를 통해 데이터를 추려내는 방식으로 코드가 동작한다.

```
// App.vue

const searchKeyword = ref('');

const onSearch = () => {
  const query = searchKeyword.value.trim();
  if (!query) return;
  router.push({ name: 'search', query: { q: query } });
};
```

- searchKeyword
 - 사용자가 검색한 영화의 이름을 담는 변수로, 입력창과 v-model로 양방향 바인딩 되어있기에 글자를 입력할 때마다 변수의 값이 갱신된다.
- onSearch()
 - 검색창에서 사용자가 엔터를 누르거나 검색 버튼을 누를 시 호출되는 함수이다.
 - query에는 사용자가 입력한 영화 제목을 trim()을 이용해 앞 뒤의 공백을 걸러낸 값을 담는다. 사용자가 공백만을 입력했을 경우, " "는 빈 문자열("")로 처리된다.
 - `if (!query) return;` 는 아무것도 입력하지 않았거나, 공백을 입력했을 경우 검색 결과 페이지로 넘어가지 않도록 한다.
 - `router.push({ name: 'search', query: { q: query } })` 는 검색어를 URL에 `/search?q=검색어` 와 같은 형태로 실어 검색 결과 페이지로 이동한다. 이를 통해 검색 결과 페이지로 사용자의 검색어를 옮길 수 있다.

```
// views/SearchView.vue

const store = useMovieStore();
const route = useRoute();

const runSearch = async () => {
  if (store.movies.length === 0) {
    await store.fetchMovies();
  }
  store.searchMovies(route.query.q);
  document.title = `🔍 '${route.query.q}' 검색 결과`;
};

onMounted(runSearch);

watch(() => route.query.q, runSearch);
```

- route
 - `useRoute()` 를 사용하여 route 변수에 현재 URL의 정보를 가져온 객체를 담는다.
- runSearch()
 - 검색 결과 페이지가 처음 렌더링 될 때 실행되는 함수로, 해당 함수를 통해 사용자에게 검색 결과를 제공한다.
 - 만약 검색 결과를 가져오고자 하는 movies의 배열에 요소가 없다면, 비동기 함수 `await`를 사용하여 영화 목록을 fetch하여 오류를 방지한다.
 - `store.searchMovies(route.query.q);` query에 담겨있던 검색어(q)를 store의 검색 메서드에 넘겨 찾고자 하는 영화의 필터링을 수행한다. 결과는 store의 `searchResults`에 채워진다.
 - 페이지가 그대로인데 검색어만 바뀌는 경우에는 기존의 검색 결과가 계속 유지되는 문제가 발생한다. 따라서 `watch(() => route.query.q, runSearch)` 를 통해, vue가 `route.query.q` 데이터의 변화를 추적하다가 값이 바뀌면 `runSearch`를 다시 실행해 화면을 새롭게 렌더링하도록 한다.

```
// stores/movieStore.js

const searchResults = ref([]);

const searchMovies = (keyword) => {
  const query = (keyword || "").trim().toLowerCase();
  searchResults.value = movies.value.filter((movie) =>
    movie.title.toLowerCase().includes(query),
  );
};
```

- searchResults
 - 검색 결과를 담는 반응형 배열이다.
- searchMovies()
 - keyword(검색어)를 인자로 받는다. 검색어가 undefined로 넘어올 경우 값을 ""로 대체하고, 정확한 검색 결과 제공을 위해 앞뒤 공백을 제거하고 검색어를 모두 소문자로 만든다.
 - `movies.value.filter((movie) => movie.title.toLowerCase().includes(query),);`
 filter은 조건이 true인 경우에만 해당 값을 포함한 새로운 배열을 반환하는 함수이다. 영화 리스트 중에 사용자가 입력한 키워드가 포함된 영화 제목이 있을 경우(대소문자 일치를 위해 toLowerCase() 또한 적용해준다.) 해당 영화들의 배열을 searchResults의 값으로 담는다.

```
// views/SearchView.vue

<main class="page">
  <div class="header-section">
    <h1>🔍 '{{ route.query.q }}' 검색 결과</h1>
    <p class="sub-title">
      총 {{ store.searchResults.length }}건의 영화를 찾았습니다.
    </p>
  </div>
```

```

... 생략
<div v-else class="movie-list">
  <div
    v-for="movie in store.searchResults"
    :key="movie.id"
    class="movie-card"
  >
    ... 생략 => 기존 영화 카드를 불러내는 코드와 유사. 대신 store
    을 searchResults로 사용
  </div>
</div>
</main>

```

◦ 검색 결과 페이지

- `route.query.q` 를 통해 사용자가 입력한 검색어를 불러온다. (/search?q=검색어) 이후, 앞선 검색 및 필터링 로직을 거친 store의 searchResults를 가져와서 화면에 렌더링한다.

• 추가 미션 [선택 3] - 찜 목록

```

// stores/movieStore.js

const favorites = ref(JSON.parse(sessionStorage.getItem("favorites")) || []);

```

◦ favorites

- sessionStorage로부터 찜한 영화들을 가져와 담는 반응형 배열이다.
- `JSON.parse()` 를 통해 세션 스토리지에 저장된 문자열을 배열 객체로 파싱한다.
- 읽어들인 값이 null 일 때를 방지하여 || 조건을 추가한다.

```

// stores/movieStore.js

const toggleFavorite = (movieId) => {
  const movie = movies.value.find((m) => m.id === movieId

```

```

d);
  if (movie) {
    movie.isFavorite = !movie.isFavorite;
    if (movie.isFavorite) {
      favorites.value.push(movie);
    } else {
      favorites.value = favorites.value.filter((m) => m.id
!== movieId);
    }
    sessionStorage.setItem("favorites", JSON.stringify(fav
orites.value));
  }
};

```

◦ toggleFavorite()

- find 는 movies 배열을 훑어 id가 일치하는 영화를 찾아 반환한다. find의 경우에는 filter와 달리 객체 한 개를 반환한다.
- 반환된 객체가 있다면, movie.isFavorite의 값을 반대로 뒤집는다. (true를 false로, false를 true로)
- 꺾을 때(isFavorite가 true일 때)에는 push를 통해 배열 끝에 객체를 추가한다.
- 꺾을 해제할 때에는 filter를 통해 해당 id에 해당하는 영화를 제외한 새로운 배열을 만든다.
- `sessionStorage.setItem("favorites", JSON.stringify(favorites.value));` 에서는 객체를 다시 문자열로 변환해서 세션 스토리지에 저장한다.

```

const removeFavorite = (movieId) => {
  favorites.value = favorites.value.filter((m) => m.id !==
movieId);
  const movie = movies.value.find((m) => m.id === movieI
d);
  if (movie) {
    movie.isFavorite = false;
  }
}

```

```
sessionStorage.setItem("favorites", JSON.stringify(favorites.value));
};
```

◦ removeFavorite()

- 찜 목록 조회 페이지에 진입했을 때, movies 배열이 비어있을 경우를 대비하여 별도의 함수를 준비한다. movies 배열이 비어있다면 toggleFavorite는 find로 영화를 찾지 못해 동작하지 못한다.
- 따라서 favorites 배열에 filter를 통해 사용자의 찜 목록을 담은 새로운 배열을 만든다.
- 그 다음 movies에 같은 영화가 있을 때만 isFavorite를 false로 만든다. 이를 통해 영화 목록 페이지로 돌아갔을 때 찜의 상태가 어긋나지 않게 맞춰준다.

```
// views/FavoritesView.vue
```

```
<div v-if="store.favorites.length === 0" class="status-message empty">
  ♥ 아직 찜한 영화가 없습니다. 마음에 드는 작품을 찜해보세요!
</div>
<div v-else class="movie-list">
  <div v-for="movie in store.favorites" :key="movie.id" class="movie-card">
    ... 생략
    <button @click="store.removeFavorite(movie.id)" class="fav-btn active">
      ♥ 찜 해제
    </button>
  </div>
</div>
```

- v-if 문을 사용하여 `store.favorites.length` 의 길이에 따라 찜한 작품이 없다는 안내를 렌더링 할 지, 사용자가 찜해둔 영화의 카드들을 렌더링할지를 분기한다.
- `@click="store.removeFavorite(movie.id)"` 는 클릭을 통해 removeFavorite를 호출해서 v-for 재렌더링을 유도한다.

• 추가 미션 [선택 4] - 페이지네이션

```
// views/MoviesView.vue

const currentPage = ref(1);
const pageMovies = 8;
```

◦ currentPage

- 현재 사용자가 보고 있는 페이지가 몇 페이지인지를 담는 반응형 변수이다.
- 초기값은 1로 설정하여 페이지 진입 시 1페이지가 기본이 된다.

◦ pageMovies

- 한 페이지에 보여줄 영화 수이다. 바뀌지 않는 상수로 설정하기 위해 ref()는 생략하며, 기본 값은 8로 설정한다.

```
// views/MoviesView.vue

const getPagedMovies = () => {
  const start = (currentPage.value - 1) * pageMovies;
  const end = start + pageMovies;
  return store.movies.slice(start, end);
};
```

◦ getPagedMovies()

- 페이지네이션을 구현하기 위한 핵심 코드로, pageMovies에서 설정한 영화의 갯수 만큼 movies의 구간을 자르는 데에 사용된다.
- start는 자르기 시작할 인덱스고, end는 start + 8로 자르기를 멈출 인덱스이다.
- `slice(start, end)` 는 movies의 해당 구간을 자른 뒤, 새로운 배열을 반환하는 코드이다. (인플레이스 방식이 아니다.)
- 만약 사용자가 보고있는 페이지가 1 페이지라면 1번째~8번째 영화를 가져오고, 보고있는 페이지가 2페이지라면 9번째~16번째 영화를 가져온다.

```
// views/MoviesView.vue
```

```
const getTotalPages = () => {  
  return Math.ceil(store.movies.length / pageMovies);  
};
```

◦ getTotalPages()

- 페이지의 개수를 구하는 함수이다.
- `store.movies.length / pageMovies` 는 전체 영화 수를 한 페이지에 보여줄 영화의 수로 나눈 값이다. 하지만 딱 나누어지지 않을 경우를 고려하여, 올림을 한다. (그래야 나머지로 남는 영화들이 마지막 페이지에 담긴다.)

```
// views/MoviesView.vue
```

```
const getPageNumbers = () => {  
  const pages = [];  
  for (let i = 1; i <= getTotalPages(); i++) {  
    pages.push(i);  
  }  
  return pages;  
};
```

◦ getPageNumbers()

- 빈 배열 `pages`에서 시작해, 1부터 총 페이지수까지 반복하며 `push`로 숫자를 채운다.
- 총 페이지 수에 따라 숫자 배열이 만들어진다. 총 페이지가 3이면 `[1, 2, 3]`이라는 배열이 만들어진다.
- 이 배열을 템플릿에서 `v-for`로 돌려 하단에 버튼을 그린다.

```
// views/MoviesView.vue
```

```
const goToPage = (page) => {  
  currentPage.value = page;  
};
```



```
window.scrollTo({ top: 0, behavior: "smooth" });  
};
```

- goToPage()

- 사용자가 버튼을 클릭하면 그 번호가 함수의 인자 page로 넘어온다.
- `currentPage.value = page` 로 사용자가 선택한 page로 교체한다. 이와 같이 `currentPage`의 value가 교체되면, `currentPage`가 반응형이기에 값이 바뀌는 순간 페이지는 다시 재렌더링 된다. (`getPagedMovies()`가 다시 호출된다.)

```
// views/MoviesView.vue  
  
watch(  
  () => store.sortBy,  
  () => {  
    currentPage.value = 1;  
  },  
);
```

- 정렬과의 동기화를 위해 `watch()`를 추가한다.
- `store.sortBy`(정렬 기준) 값의 변화를 추적하고, 정렬 기준이 바뀌게 된다면 현재 페이지를 1로 바꾸어서 정렬된 값을 사전 순, 혹은 내림차 순으로 처음부터 확인해 볼 수 있도록 한다.